

Big Data Analysis of Chicago Crime Records

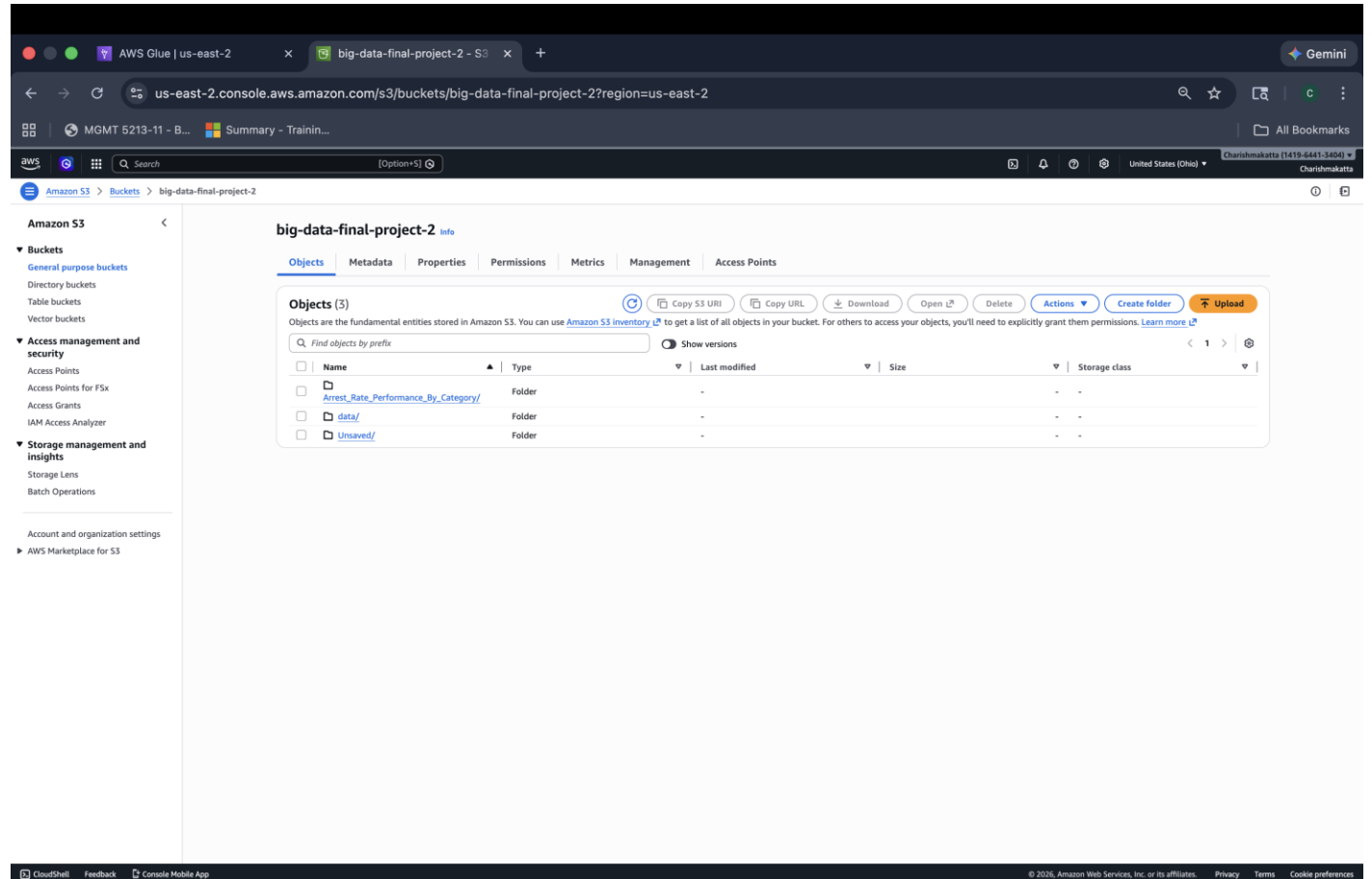
How effectively data is analyzed stored and processed using different cloud based tools.

BIG DATA AND CLOUD COMPUTING –IT 5613

Presented by : CHARISHMA KATTA

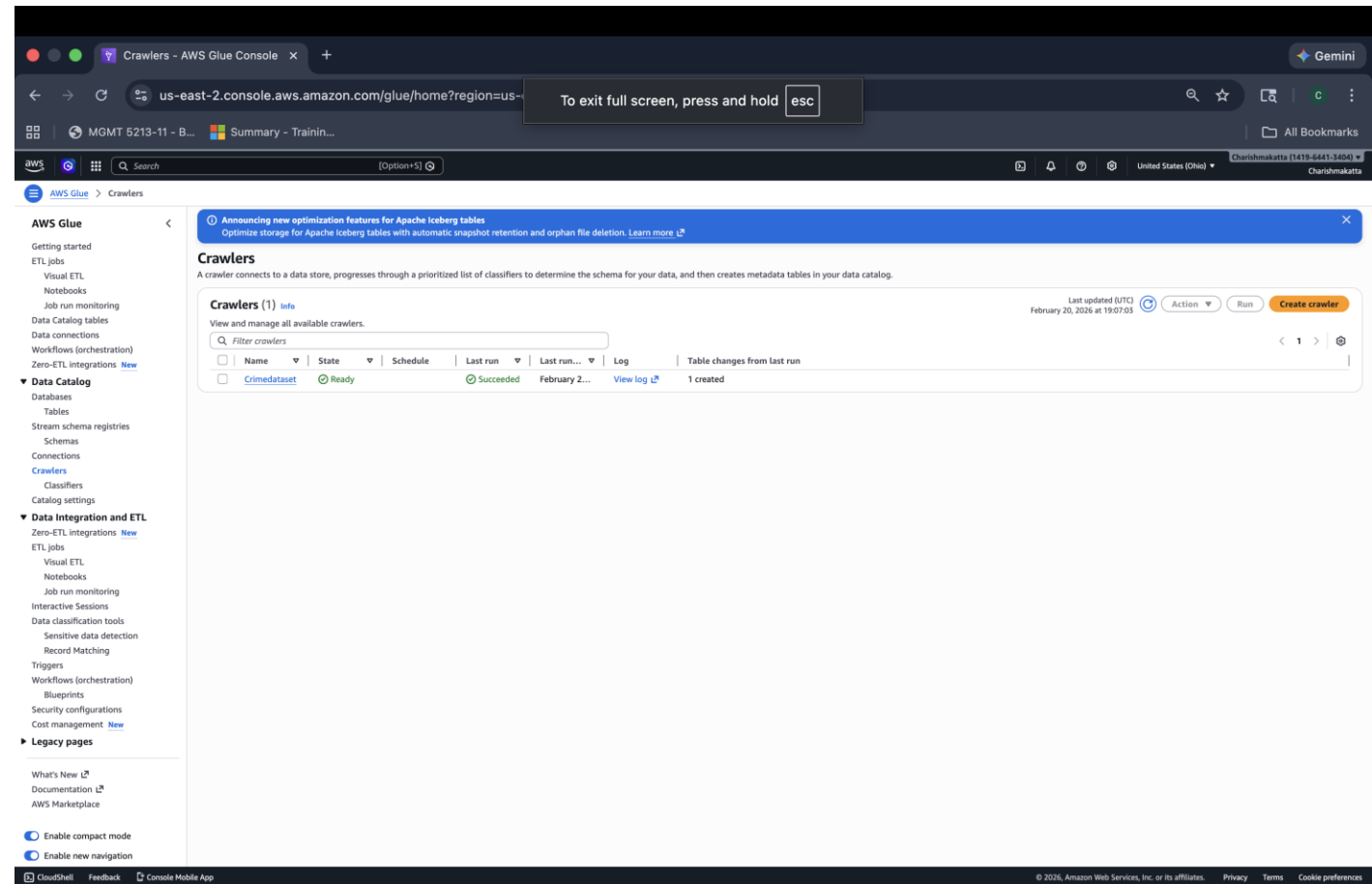
Data Storage & The Data Lake Concept

- S3 is used here because it is **highly scalable** and it allows for **Decoupled storage** and compute.
- A data lake is created here using Amazon S3 and the data csv file is uploaded into the bucket.



Metadata Discovery & Automation

- This is the **Discovery** phase. Glue identifies the column headers and data types.
- AWS services know how to read the raw file without us having to move the data.



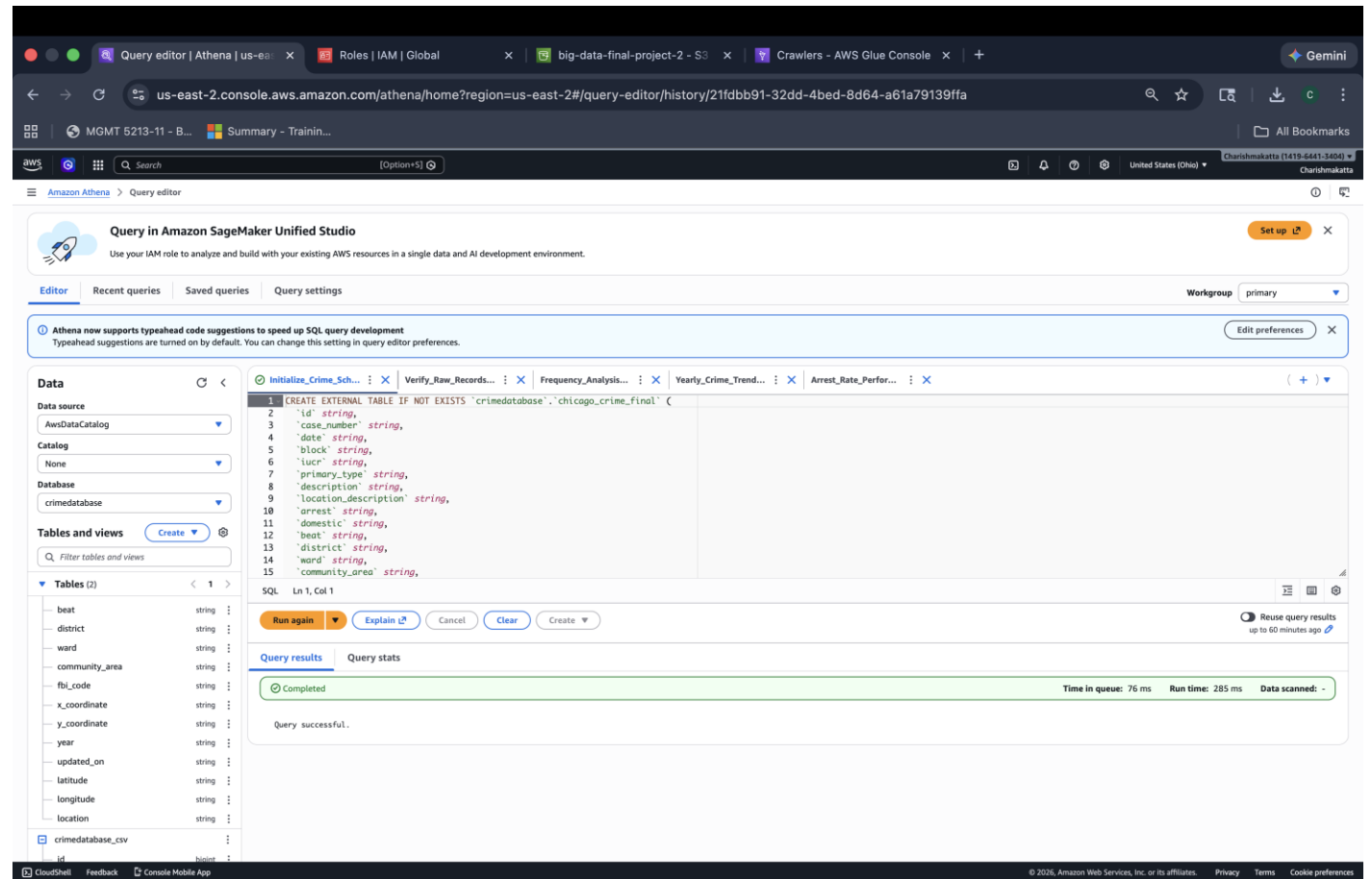
Troubleshooting Real-World Data

- As seen in the screenshot, the initial query returned technical metadata (words like hive, bigint, varchar) instead of crime records.
- The Crawler accidentally indexed hidden system files instead of just the CSV.

The screenshot shows the Amazon Athena console interface. At the top, there's a navigation bar with the AWS logo and the text "Amazon Athena" and "Query editor". Below this, there's a sidebar on the left with sections for "Data source" (set to "AwsDataCatalog"), "Catalog" (set to "None"), "Database" (set to "crimdatabase"), and "Tables and views" (with a "Create" button and a search filter). The main area displays the query editor with the SQL statement: `SELECT * FROM chicago_crime_fixed LIMIT 10`. Below the query, there are buttons for "Run again", "Explain", "Cancel", "Clear", and "Create". The "Query results" section shows a green bar indicating the query is "Completed". Below this, there's a table of results with 10 rows, each containing metadata for the 'chicago_crime_fixed' table. The columns in the results table are: #, id, case_number, date, block, lucr, primary_type, description, location_description, arrest, domestic, beat. The first row of data is: 1, 20260220_183016_00025_jhrbe. The console also shows various status indicators and statistics at the bottom of the query results section, such as "Time in queue: 94 ms", "Run time: 23.61 sec", and "Data scanned: 2.55 KB".

Data Engineering & Serialization

- I wrote a custom **SQL DDL script** to manually create an "External Table." I used the **OpenCSVSerde** library.
- I defined all columns as **STRING**. This was a strategic choice to handle "**dirty data**" (like empty coordinate fields) that was causing numeric parsing errors earlier.



Data Validation

- I ran a **SELECT * LIMIT 10** query.
- This screenshot proves the fix worked! You can now see actual crime descriptions like "THEFT" and "DECEPTIVE PRACTICE."
- This confirmed that our **Schema-on-Read** was correctly mapped to the physical file in S3.

The screenshot displays the Amazon Athena Query Editor interface. The query editor shows the following SQL query:

```
1 SELECT * FROM "crimedatabase"."chicago_crime_final" LIMIT 10;
```

The query has been executed successfully, and the results are displayed in a table with 10 rows. The table columns are: id, case_number, date, block, lucr, primary_type, description, location_description, and arrest. The results show various crime incidents, including "DECEPTIVE PRACTICE", "MOTOR VEHICLE THEFT", "BATTERY", "ASSAULT", "THEFT", "CRIMINAL DAMAGE", and "CRIMINAL TRESPASS".

#	id	case_number	date	block	lucr	primary_type	description	location_description	arrest
1	14109442	JK147827	02/11/2026 12:00:00 AM	042XX W MONROE ST	1130	DECEPTIVE PRACTICE	FRAUD OR CONFIDENCE GAME	OTHER (SPECIFY)	false
2	14107378	JK145300	02/11/2026 12:00:00 AM	027XX W WASHINGTON BLVD	0910	MOTOR VEHICLE THEFT	AUTOMOBILE	STREET	false
3	14107897	JK145843	02/11/2026 12:00:00 AM	017XX N MILWAUKEE AVE	1130	DECEPTIVE PRACTICE	FRAUD OR CONFIDENCE GAME	GROCERY FOOD STORE	false
4	14107923	JK146147	02/11/2026 12:00:00 AM	083XX S HALSTED ST	0460	BATTERY	SIMPLE	PARKING LOT / GARAGE (NON RESIDENTIAL)	false
5	14108101	JK146028	02/11/2026 12:00:00 AM	029XX W 40TH PL	0520	ASSAULT	AGGRAVATED - KNIFE / CUTTING INSTRUMENT	APARTMENT	true
6	14108180	JK146306	02/11/2026 12:00:00 AM	054XX S KEDZIE AVE	0860	THEFT	RETAIL THEFT	SMALL RETAIL STORE	false
7	14108261	JK146372	02/11/2026 12:00:00 AM	079XX S AVALON AVE	1320	CRIMINAL DAMAGE	TO VEHICLE	STREET	false
8	14108674	JK146974	02/11/2026 12:00:00 AM	065XX S HERMITAGE AVE	1365	CRIMINAL TRESPASS	TO RESIDENCE	APARTMENT	true
9	14114262	JK153734	02/11/2026 12:00:00 AM	028XX W MC LEAN AVE	1305	CRIMINAL DAMAGE	CRIMINAL DEFAACEMENT	RESIDENCE	false
10	14107651	JK145627	02/10/2026 11:55:00 PM	051XX W BLOOMINGDALE AVE	1310	CRIMINAL DAMAGE	TO PROPERTY	RESIDENCE	false

Frequency & Density Analysis

- I used the **COUNT** and **GROUP BY** functions to aggregate the data by crime type.
- This analysis identifies where the most pressure is on city resources. We discovered that **Theft** and **Battery** are the most frequent crimes in the dataset, totaling millions of records.

The screenshot displays the Amazon Athena Query Editor interface. The SQL query being executed is:

```
1 SELECT "primary_type", COUNT(*) AS total_reports
2 FROM "crimedatabase"."chicago_crime_final"
3 GROUP BY "primary_type"
4 ORDER BY total_reports DESC;
```

The query results are shown in a table with 34 rows. The top results are:

#	primary_type	total_reports
1	THEFT	1804952
2	BATTERY	1547900
3	CRIMINAL DAMAGE	966131
4	NARCOTICS	765924
5	ASSAULT	570715
6	OTHER OFFENSE	530466
7	BURGLARY	448831
8	MOTOR VEHICLE THEFT	436600
9	DECEPTIVE PRACTICE	393124

The interface also shows a sidebar with data sources and tables, and a bottom status bar indicating the query is completed with a run time of 2.521 seconds and 2.20 GB of data scanned.

Temporal Analysis (Time Series)

- I analyzed the crime volume per year from 2001 to the present.
- This shows the "Big Data" trend over time. It allows us to see if crime is trending up or down, which is essential for high-level policy making and historical reporting.

The screenshot displays the Amazon Athena Query Editor interface. The SQL query being executed is:

```
1 SELECT "year", COUNT(*) AS crime_count
2 FROM "crimedatabase"."chicago_crime_final"
3 GROUP BY "year"
4 ORDER BY "year" ASC;
```

The query results are shown in a table with 26 rows. The columns are 'year' and 'crime_count'.

#	year	crime_count
1	2001	485958
2	2002	486831
3	2003	475999
4	2004	469443
5	2005	453788
6	2006	448198
7	2007	437108
8	2008	427216
9	2009	392866

The interface also shows the 'Data' sidebar on the left with a list of tables and views, including 'beat', 'district', 'ward', 'community_area', 'fbi_code', 'x_coordinate', 'y_coordinate', 'year', 'updated_on', 'latitude', 'longitude', 'location', 'crimedatabase_csv', 'id', 'case number', 'date', 'block', 'lucr', and 'primary type'. The 'Query results' section at the bottom indicates the query is 'Completed' with a run time of 2.207 seconds and 2.20 GB of data scanned.

Performance Metrics (KPIs)

- I calculated the **Arrest Rate** percentage for each crime type using **CAST** and **SUM** logic.
- This is a key effectiveness metric. While some crimes like "**Narcotics**" have a near 100% arrest rate, others are much lower.
- This identifies where law enforcement is most (and least) successful in closing cases.

The screenshot displays the Amazon Athena Query Editor interface. The SQL query in the editor calculates the arrest rate for various crime types by dividing the total number of arrests by the total number of crimes, then casting the result to a double and multiplying by 100. The query is executed, and the results are shown in a table.

SQL Query:

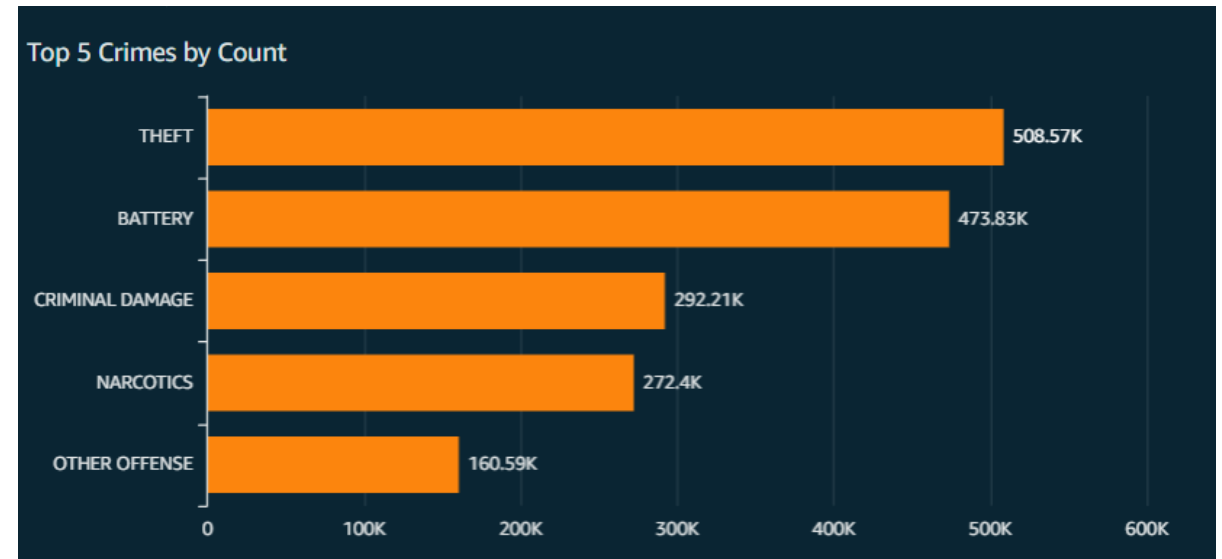
```
1 SELECT
2   "primary_type",
3   COUNT(*) AS total_crimes,
4   SUM(CASE WHEN "arrest" = "true" THEN 1 ELSE 0 END) AS total_arrests,
5   (CAST(SUM(CASE WHEN "arrest" = "true" THEN 1 ELSE 0 END) AS DOUBLE) / COUNT(*)) * 100 AS arrest_percentage
6 FROM "crimedbatabase"."chicago_crime_final"
7 GROUP BY "primary_type"
8 ORDER BY arrest_percentage DESC;
```

Query Results (34):

#	primary_type	total_crimes	total_arrests	arrest_percentage
1	DOMESTIC VIOLENCE	1	1	100.0
2	PROSTITUTION	70469	70156	99.55583306134612
3	NARCOTICS	765924	760795	99.33035131422962
4	GAMBLING	14666	14559	99.27042138279013
5	LIQUOR LAW VIOLATION	15446	15291	98.99650394924252
6	PUBLIC INDECENCY	223	219	98.20627802690582
7	CONCEALED CARRY LICENSE VIOLATION	1711	1654	96.66861484511982
8	INTERFERENCE WITH PUBLIC OFFICER	20507	18799	91.67113668503438
9	OBSCENITY	971	713	73.42945417095777

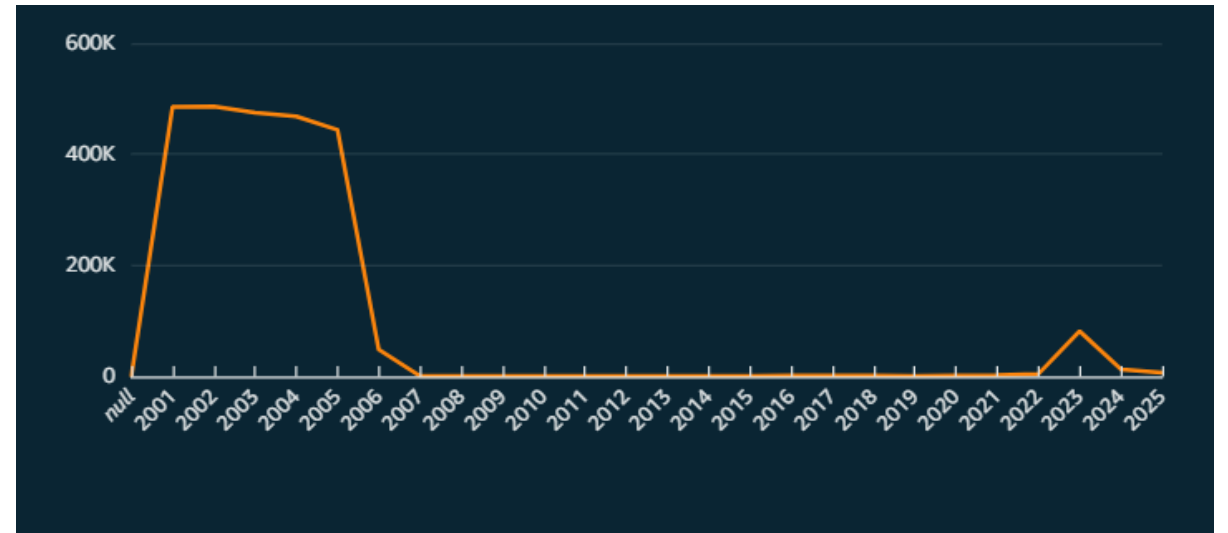
Data Visualization

Bar Chart (Top Crimes)



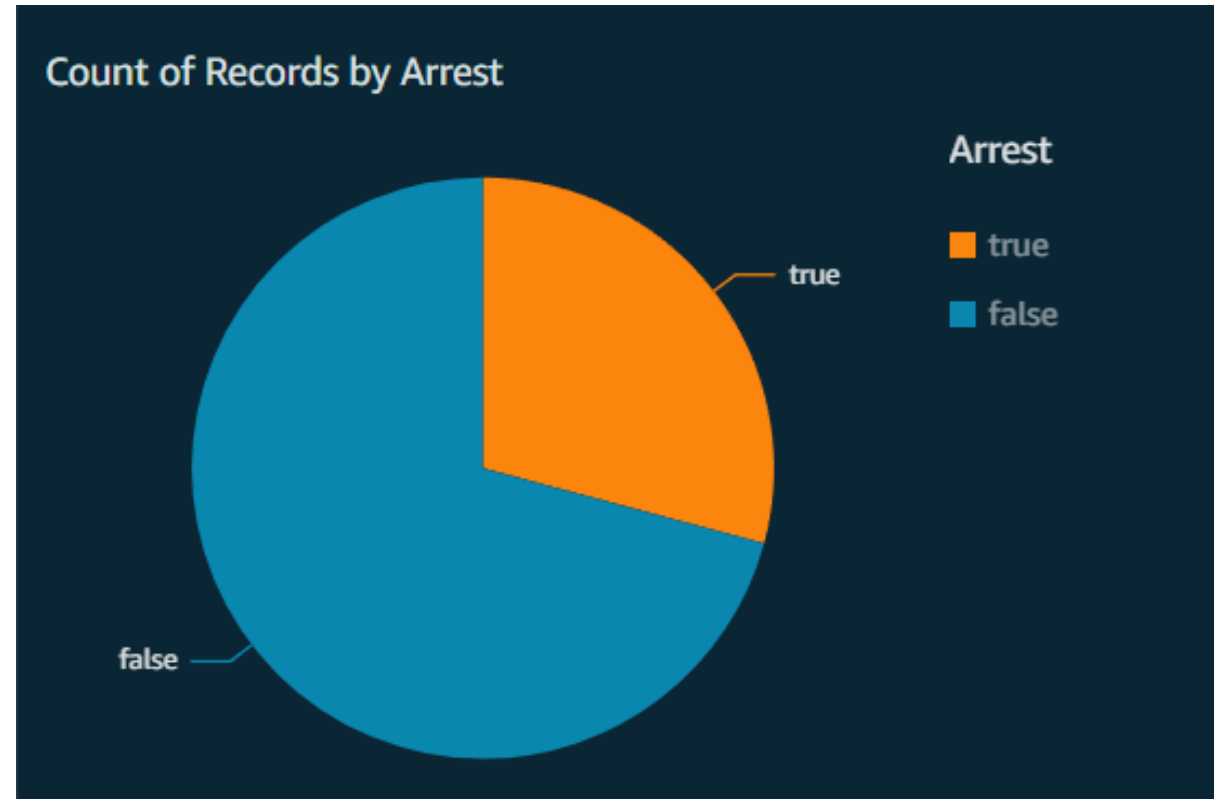
Data Visualization

Line Chart (Crimes by Year)



Data Visualization

PIE Chart (Total Arrests)



Conclusion: Serverless Analytics

- We successfully built a **Serverless Pipeline**.
- By using **S3, Glue, and Athena**, we processed millions of rows of data without ever having to manage a single server or database instance.

The screenshot displays the Amazon Athena console landing page in a web browser. The browser's address bar shows the URL `us-east-2.console.aws.amazon.com/athena/home?region=us-east-2#/landing-page`. The page features a navigation bar with the AWS logo and a search bar. The main content area is titled 'How it works' and contains a diagram illustrating the Athena workflow:

- 1. Point to your data source**: Register and select any data source. (Icon: S3 bucket)
- 2. Start querying**: Athena's query editor lets you author and run SQL statements to query your data source. (Icon: Code editor)
- 3. Analyze the results**: Query results are delivered within seconds. (Icon: Magnifying glass over a document)

The diagram shows 'Amazon Athena' at the center, connected to the data source and the query editor. Below the diagram, the 'Benefits and features' section highlights:

- Start querying now**: Athena is serverless. You can quickly query your data without having to set up and manage any servers or data warehouses.
- Powerful**: Athena is ideal for quick, ad-hoc querying, but it can also handle complex analysis, including large joins, window functions, and arrays.

On the right side of the page, there are three panels:

- Pricing**: Region: US East (Ohio). SQL queries per TB scanned: \$5.00 per TB. SQL queries on Provisioned Capacity: \$0.30 per DPU hour. Apache Spark calculations: \$0.35 per DPU hour. [Cost calculator](#)
- Getting started**: Add the data source and identify the table and column names. Then you can begin querying your data right away. [Learn more](#)
- More resources**: [Documentation](#), [API reference](#), [FAQs](#), [Support forums](#), [Release notes](#)

The footer of the page includes links for CloudShell, Feedback, Console Mobile App, and copyright information: © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences.

FUTURE WORK

